

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	Total Marks:	25
CO2 Statement:	Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills		
Student Name:	Lella Thannveer Reddy	Reg. No.:	192524293
Date:	03/08/2026	Duration:	60 minutes

Code of Conduct

I Lella Thannveer Reddy/192524293 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Lella Thannveer Reddy

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:

Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

- Study and Analyze the Scenario** ○ Carefully understand the given problem statement.
 - Identify the business domain, stakeholders, existing challenges, and system objectives.
- Identify Functional and Non-Functional Requirements** ○ List all functional requirements required by the proposed system.
 - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
- Identify Actors and Use Cases** ○ Identify all primary and secondary actors interacting with the system. ○ List the major use cases associated with each actor.
 - Draw a Use Case Diagram representing the interactions between actors and the system.

4. **Prepare the Software Requirement Specification (SRS)** Develop an SRS document containing:
 - Introduction ○ Problem Description ○ Scope of the System ○ Functional Requirements
 - Non-Functional Requirements ○ Use Case Descriptions ○ Data Requirements ○ External Interface Requirements ○ System Features
5. **Identify Assumptions and Constraints** ○ List the assumptions considered while developing the system.
 - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency** ○ Verify whether all stakeholder requirements have been addressed. ○ Check for ambiguity, redundancy, inconsistency, and missing requirements.
 - Suggest improvements to enhance the quality and completeness of the requirement specification.

1. Study and Analyze the Scenario

Business Domain:

The proposed software system belongs to a generic information management/business application domain where users perform daily operations through a centralized system.

Stakeholders:

- End Users
- System Administrator
- Organization/Management
- Database Administrator
- Developers and Maintenance Team

Existing Challenges:

- Manual data management
- Duplicate and inconsistent records
- Slow processing and report generation
- Data security risks
- Difficulty in tracking information

System Objectives:

- Automate business processes.
- Improve accuracy and efficiency.
- Provide secure and reliable data management.
- Generate reports quickly.
- Improve user satisfaction and decision-making.

2. Functional and Non-Functional Requirements

Functional Requirements

- User registration and login
- User authentication and authorization
- Create, Read, Update, Delete (CRUD) operations
- Search and filter records
- Report generation
- Notifications
- Audit logs

- Database backup and recovery

Non-Functional Requirements

- Performance: Fast response time.
- Security: Authentication, authorization, encryption.
- Reliability: Accurate processing with fault tolerance.
- Usability: Simple and user-friendly interface.
- Scalability: Supports increasing users and data.
- Maintainability: Modular and well-documented design.
- Availability: High uptime with backup support.

3. Identify Actors and Use Cases

Primary Actors

- Administrator
- Registered User

Secondary Actors

- Database
- Email/SMS Service
- External API

Major Use Cases

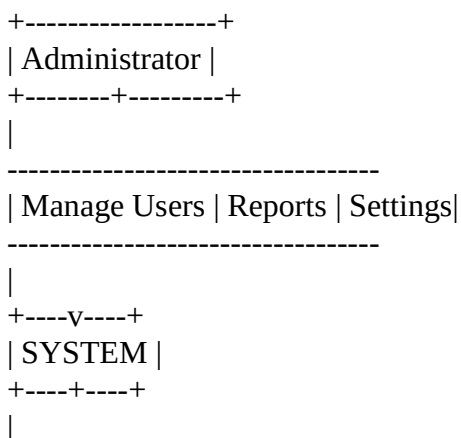
Administrator:

- Manage users
- Manage records
- Generate reports
- View logs
- Configure system

User:

- Register/Login
- Manage profile
- Add, edit, delete, and search data
- View reports
- Logout

Simple Use Case Diagram



```
-----
| Login | CRUD | Search | Reports |
-----
|
+-----+-----+
| User |
+-----+
```

4. Software Requirement Specification (SRS)

Introduction

This document specifies the functional and non-functional requirements of the proposed software system.

Problem Description

Current manual processes are slow, error-prone, and difficult to maintain. A computerized solution is required.

Scope

The system manages users, records, reports, and secure data storage while providing an easy-to-use interface.

Functional Requirements

- User authentication
- Data management
- Search
- Reporting
- Notifications

Non-Functional Requirements

- Security
- Performance
- Reliability
- Scalability
- Maintainability
- Availability

Use Case Description (Example: Login)

Actor: User

Precondition: User account exists.

Main Flow: Enter credentials → Validate → Access granted.

Postcondition: User dashboard displayed.

Data Requirements

- User information
- Transaction/Record data
- Reports
- Logs
- Configuration data

External Interface Requirements

- User Interface (Web/Mobile)
- Database (MySQL/PostgreSQL)
- REST APIs
- Email/SMS gateway

System Features

- Secure login
- Record management
- Search
- Dashboard
- Reporting
- Backup and recovery

5. Assumptions and Constraints

Assumptions

- Users have internet access.
- Database server is available.
- Users possess valid credentials.
- Required hardware and software are installed.

Constraints

Technical:

- Limited hardware resources.
- Software compatibility.

Operational:

- User training may be required.

Economic:

- Budget limitations.

Legal:

- Data privacy and security regulations.

Time:

- Project must be completed within the allocated schedule.

6. Validate Requirement Completeness and Consistency

Validation

- All stakeholder requirements have been identified.
- Functional and non-functional requirements are documented.
- Use cases cover major business processes.

Consistency Check

- No conflicting requirements.
- No duplicate requirements.
- Requirements are clear and measurable.
- Missing requirements should be reviewed with stakeholders.

Suggested Improvements

- Conduct stakeholder review meetings.
- Prioritize requirements using MoSCoW.
- Add detailed acceptance criteria.
- Include risk analysis and traceability matrix.
- Update the SRS whenever requirements change.

Conclusion

The proposed SRS provides a complete foundation for software design, implementation, testing, and maintenance. Proper validation ensures that the software satisfies stakeholder expectations and supports future enhancements.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification	Clearly identifies comprehensive, accurate, and well classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling	Correctly identify all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS) Documentation	SRS is well structured, complete, professionally organized, and includes all required	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or lacks essential

	sections.			components.
Assumptions, Constraints, and Requirement Validation	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	Mark
Scenario Understanding and Problem Analysis	/5
Functional and Non-Functional Requirement Identification	/5
Actor Identification and Use Case Modelling	/5
Software Requirement Specification (SRS) Documentation	/5
Assumptions, Constraints, and Requirement Validation	/5
TOTAL	/5